

Chương 1:

Giới thiệu DỮ LIỆU LỚN

*Nguồn tham khảo (dịch + trình bày lại):
Slides Khoa học dữ liệu lớn – Phùng Quốc Định)
Slides Data-Intensive Text Processing with MapReduce*

Nguyễn Ngọc Điệp

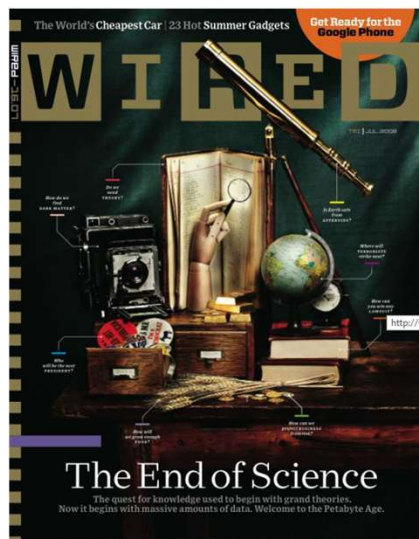
Nội dung

- Dữ liệu lớn là gì?
 - Dữ liệu lớn từ đâu đến?
 - Cơ hội và thách thức của dữ liệu lớn
- Tiếp cận dữ liệu lớn như thế nào?
 - Quản lý dữ liệu lớn
 - Xử lý dữ liệu lớn
 - Tính toán phân tán và song song
 - Giới thiệu nền tảng công nghệ

Nguyễn Ngọc Điệp

Dữ liệu LỚN là gì?

Nguyễn Ngọc Diệp



“Chúng ta thường xây dựng lý thuyết trước khi khai phá kiến thức. Nhưng ngày nay, việc này thường lại bắt đầu từ dữ liệu trước. Thời đại dữ liệu lớn đã bắt đầu!”



Kết nối vạn vật



Tính toán đám mây

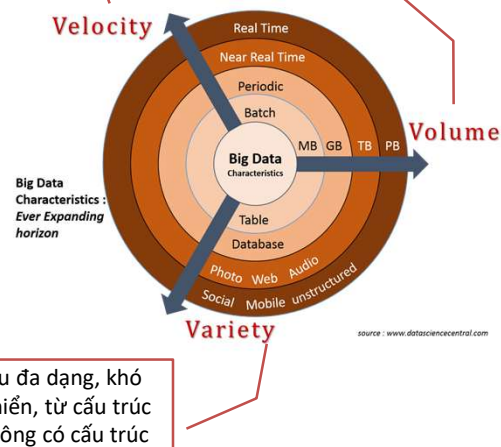
Nguyễn Ngọc Diệp

Dữ liệu lớn là gì?

- DLL là các tập dữ liệu rất lớn và/hoặc rất phức tạp
- Vượt quá khả năng kỹ thuật và lý thuyết truyền thống
- DLL có ba đặc điểm quan trọng (3Vs)

Dòng dữ liệu không ngừng chuyển động

.....
Zettabytes (10^{18})
Petabytes (10^{15})



Dữ liệu đa dạng, khó điều khiển, từ cấu trúc đến không có cấu trúc

Nguyễn Ngọc Diệp

Dữ liệu lớn là gì?

Một zettabyte lớn thế nào?



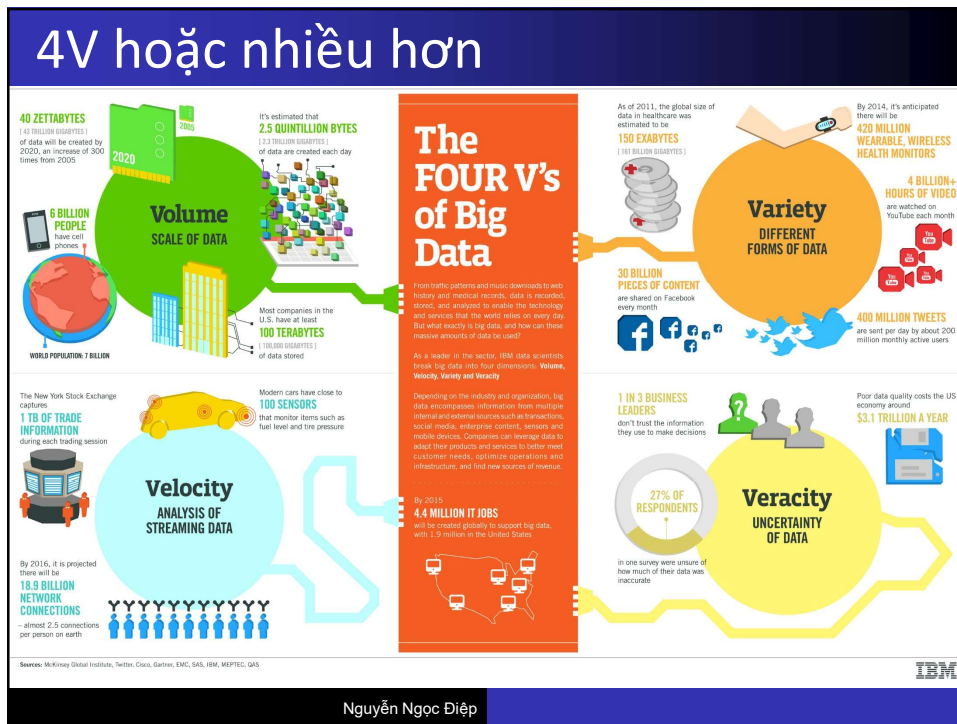
If the Digital Universe were represented by the memory in a stack of tablets, in 2013 it would have stretched two-thirds the way to the Moon*

By 2020, there would be 6.6 stacks from the Earth to the Moon*

Dự tính đến năm 2020 dữ liệu toàn cầu sẽ đạt khoảng 44 ZB.

Nếu ta dùng nhiều máy iPad để chứa số dữ liệu này và chồng lên nhau, chúng sẽ lấp đầy hơn sáu lần khoảng cách từ trái đất đến mặt trăng.

Nguyễn Ngọc Diệp



Dữ liệu lớn từ đâu đến?

Dữ liệu lớn đến từ đâu?



"The average person today processes more data in a single day than a person in the 1500's did in an entire life time"

Nguồn: Smolan and Ervitt, The human face of big data, 2013]

Every 60 seconds

- 98,000+ tweets
- 695,000 status updates
- 11 million instant messages
- 698,445 Google searches
- 168 million+ emails sent
- 1,820TB of data created
- 217 new mobile web users

Nguyễn Ngọc Diệp

Dữ liệu lớn đến từ đâu?

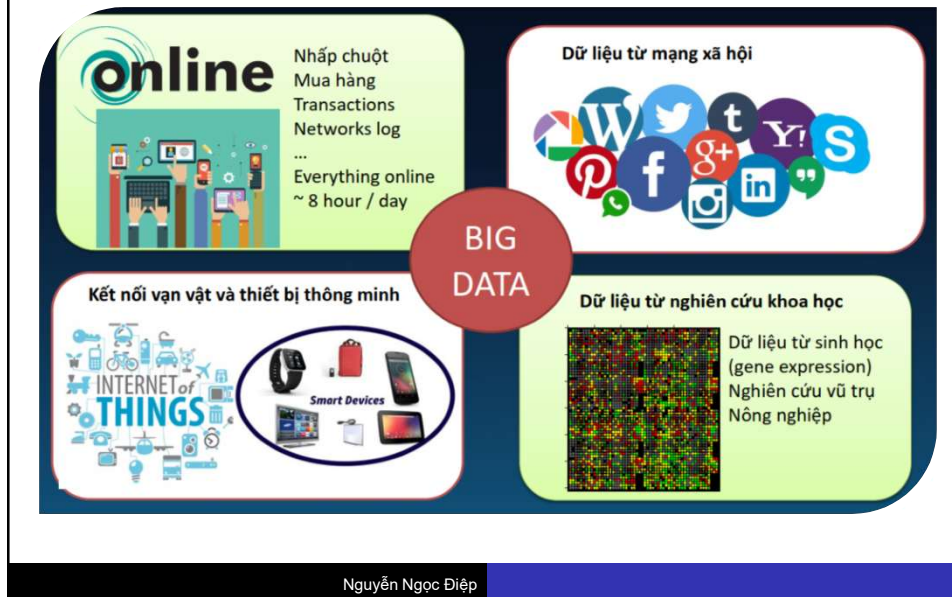
"Chỉ trong ngày đầu tiên một em bé sinh ra đời, số lượng dữ liệu thu thập được tương đương với 70 lần thông tin trong Thư viện Quốc hội Mỹ"



DURING THE FIRST DAY OF A BABY'S LIFE, THE AMOUNT OF DATA GENERATED BY HUMANITY IS EQUIVALENT TO 70 TIMES THE INFORMATION CONTAINED IN THE LIBRARY OF CONGRESS. ▼

Nguyễn Ngọc Diệp

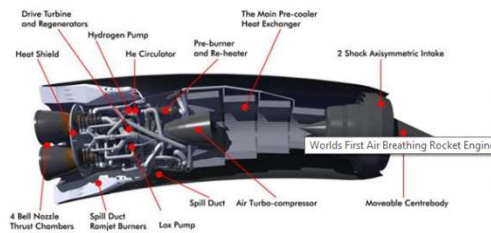
Dữ liệu lớn đến từ đâu?



Dữ liệu lớn đến từ đâu?

- “Lớn mà không to, to mà không lớn” [GS Hồ Tú Bảo]

(tương tác dữ liệu cảm biến)



Lớn mà không to

(đếm số lượng nhấp chuột trên toàn cầu)



To mà không lớn

Lean data vs big data?
Complexity or size?

Nguyễn Ngọc Diệp

Một số ví dụ khác?

- Google xử lý 20 PB / ngày (2008)
- Wayback Machine có 3 PB + 100 TB/tháng (3/2009)
- Facebook có 2,5 PB dữ liệu người dùng + 15 TB/ngày (4/2009)
- eBay có 6,5 PB dữ liệu người dùng + 50 TB/ngày
- LHC có 5 PB một năm (??)



640K là đủ cho tất cả mọi người.

Nguyễn Ngọc Điệp

Cơ hội và thách thức

Nguyễn Ngọc Điệp

DLL có thể đem lại những cơ hội gì?

- Dữ liệu lớn và lợi ích chiến lược của quốc gia

- BQP Mỹ dành khoảng 250 triệu mỗi năm để khai thác DLL, nhằm nâng cao khả năng ra quyết định.



“Năm 2012, văn phòng chính sách khoa học và công nghệ của Mỹ thuộc Văn phòng điều hành của Tổng thống Mỹ đã công bố **84 chương trình về dữ liệu lớn thuộc 6 Bộ của Chính quyền Liên bang**. Những chương trình này đề cập đến thách thức và cơ hội của cuộc cách mạng dữ liệu lớn và **xem việc tìm lời giải cho vấn đề dữ liệu lớn là sứ mệnh** của các cơ quan chính phủ cũng như của việc cách tân và khám phá khoa học”

Nguyễn Ngọc Điệp

DLL có thể đem lại những cơ hội gì?

- Dữ liệu lớn thay đổi diện mạo doanh nghiệp, công ty công nghiệp và khởi nghiệp

- Các doanh nghiệp đã có thể truy cập tới các nguồn dữ liệu lớn:

dữ liệu độc quyền = tài nguyên

- Ngành công nghiệp cũng sẽ thay đổi mạnh mẽ.

Ví dụ: **advanced manufacturing process optimization**

- Cùng với sự phát triển mạnh mẽ của ngành khoa học dữ liệu (KHDL) là cơ hội khởi nghiệp

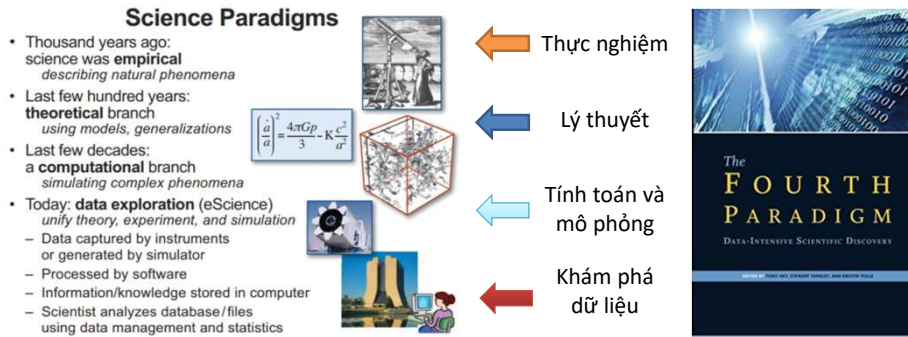
startups = ideas + KHDL + \$\$\$?



Nguyễn Ngọc Điệp

DLL mang lại lợi ích gì?

● Khám phá khoa học dựa vào dữ liệu lớn



Nguyễn Ngọc Diệp

Machine learning predicts the look of stem cells, *Nature News*, April 2017
The Allen Cell Explorer Project

"No two stem cells are identical, even if they are genetic clones Computer scientists analysed thousands of the images using deep learning programs and found relationships between the locations of cellular structures. They then used that information to predict where the structures might be when the program was given just a couple of clues, such as the position of the nucleus. The program 'learned' by comparing its predictions to actual cells"

Nguyễn Ngọc Diệp

DLL mang lại lợi ích gì?

● Khám phá khoa học dựa vào dữ liệu lớn

Science Paradigms

- Thousand years ago:
science was **empirical**
describing natural phenomena
- Last few hundred years:
theoretical branch
using models, generalizations
- Last few decades:
a **computational branch**
simulating complex phenomena
- Today: **data exploration** (eScience)
unify theory, experiment, and simulation
 - Data captured by instruments or generated by simulator
 - Processed by software
 - Information/knowledge stored in computer
 - Scientist analyzes database/files using data management and statistics

Thực nghiệm

Lý thuyết

Tính toán và mô phỏng

Khám phá dữ liệu

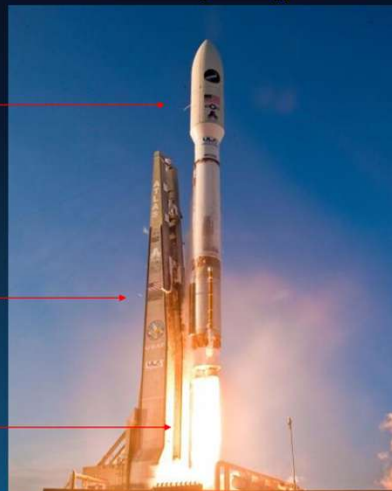
Nguyễn Ngọc Diệp

Andrew Ng's Analogy

Mô hình tính toán lớn,
e.g., deep learning/AI

Cơ sở hạ tầng tính toán,
nhà nghiên cứu,
môi trường và chính sách
như bộ phóng

Dữ liệu lớn như
nguồn năng lượng



Nguyễn Ngọc Diệp

Thách thức và vấn đề của DLL

- Dữ liệu và lưu trữ vượt quá khả năng tính toán!
 - Web, di động, cảm biến, khoa học, v.v.
 - Log hàng ngày của Facebook: 60 TB
 - 1000 dự án Gen: 200 TB
 - Chỉ mục Web của Google: 10+ PT
 - Giá của 1TB ổ đĩa lưu trữ: ~ 50\$
 - Lưu trữ ngày càng rẻ
 - Kích cỡ tăng gấp đôi sau 18 tháng
 - Cản trở do nghẽn trong tốc độ CPU và lưu trữ
 - Thời gian đọc 1 TB từ ổ đĩa: 3 giờ (100MB/s)



Cách tiếp cận và phương pháp phân tích dữ liệu trở thành chìa khóa quan trọng!

Nguyễn Ngọc Điệp

Thách thức và vấn đề của DLL

- Các vấn đề đạo đức
 - Lộ thông tin cá nhân, thu thập thông tin trộm
- An toàn thông tin và sự riêng tư
 - Dễ ăn trộm thông tin, bao gồm ăn cắp danh tính, trộm cắp thông tin quốc gia
- Các vấn đề về tấn công
 - Khai thác thông tin nhằm kiếm lợi kinh tế
- Vấn đề về quyền lực và chính trị
 - Sử dụng dữ liệu để duy trì quyền lực và ý thức hệ
- Vấn đề về sự thật
 - Duy trì sự giả dối và tuyên truyền thông tin giả
- Vấn đề về công bằng xã hội
 - Khoảng cách số có nghĩa là thông tin bị áp đảo về một nhóm khiến nhóm khác rời khỏi 'cuộc cách mạng kỹ thuật số' [Radika Gorur]

Nguyễn Ngọc Điệp

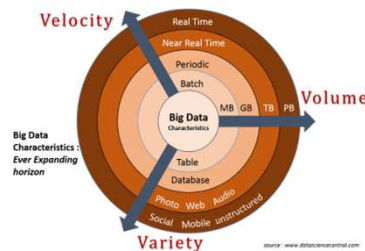
Thách thức và vấn đề của DLL

- Có phải cứ có nhiều dữ liệu thì càng tốt không?
- Điều này chưa chắc:
 - Nhầm lẫn noise/artefact với thông tin thật (more false positives).
 - Tăng giá thành lưu trữ dữ liệu và tính toán không hiệu quả.
 - Phức tạp hóa vấn đề không đúng cách.
 - Những mô hình phân tích dữ liệu tinh vi và hiệu quả có thể không ứng dụng được nữa.

Nguyễn Ngọc Diệp

Tóm tắt về DLL

- Dữ liệu lớn (DLL) xử lý tập dữ liệu rất lớn hoặc (đồng thời) rất phức tạp vượt quá giới hạn của công nghệ và kỹ thuật cổ điển.
- DLL có ba đặc tính quan trọng:
 - Kích thước rất lớn: petabytes, zettabytes
 - Dòng dữ liệu không ngừng chuyển động
 - Dữ liệu đa dạng, khó điều khiển, di chuyển từ cấu trúc (structured) sang không cấu trúc (unstructured data).
- DLL đến từ nhiều nguồn khác nhau và không ngừng lớn lên
 - Dữ liệu online, mạng xã hội.
 - Kết nối vạn vật (IoT) và thiết bị thông minh (smart devices, sensors).
 - Các giao dịch và dữ liệu trong doanh nghiệp.
- DLL đem lại nhiều cơ hội
 - Lợi ích chiến lược quốc gia
 - Doanh nghiệp và khởi nghiệp
 - Khám phá khoa học
- Nhưng cũng đặt ra nhiều thách thức và lăm cạm bẫy



Nguyễn Ngọc Diệp

BÀN LUẬN

- Câu hỏi 1: Có cần quan tâm đến dữ liệu lớn không?
- Câu hỏi 2: Có phải cứ có nhiều dữ liệu là tốt không?
- Câu hỏi 3: Dữ liệu lớn hay dữ liệu nhỏ? Lean data or big data?

Nguyễn Ngọc Diệp

Tiếp cận dữ liệu lớn như thế nào?

Nguyễn Ngọc Diệp

Chìa khóa của dữ liệu lớn

- **Đây là chìa khóa khoa học và công nghệ của DLL?**

- **Quản trị dữ liệu**, tức lưu trữ, bảo trì và truy nhập các nguồn dữ liệu lớn.



QUẢN TRỊ DỮ LIỆU

- **Phân tích dữ liệu**, tức tìm cách hiểu được dữ liệu và tìm ra các thông tin hoặc tri thức quý báu từ dữ liệu.



MÔ HÌNH HÓA DỮ LIỆU VÀ PHÂN TÍCH

- **Trao đổi, hiển thị dữ liệu** và kết quả phân tích dữ liệu để tạo ra sản phẩm hay giá trị.



HIỂN THỊ KẾT QUẢ VÀ GIÁ TRỊ

Nguyễn Ngọc Điệp

Chìa khóa của dữ liệu lớn

- **Hỏi gì khi tiếp cận DLL?**

NHỮNG VẤN ĐỀ NỀN TẢNG

- Cần lấy dữ liệu nhanh cỡ nào?
- Dữ liệu cần xử lý lớn cỡ nào?
- Mô hình xây dựng chạy một lần hay nhiều lần?



NHỮNG VẤN ĐỀ VỀ HỆ THỐNG

- Liệu có cần xử lý nhiều dữ liệu hơn nữa trong tương lai?
- Tốc độ truyền dữ liệu có quan trọng với ứng dụng không?
- Có cần xử lý lỗi phản cứng trong ứng dụng không?

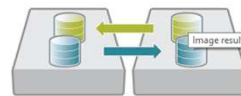
NHỮNG VẤN ĐỀ VỀ CÔNG NGHỆ

- Hạ tầng được sử dụng là gì? (cloud/ hệ thống vật lý)
- Công nghệ được dùng để xử lý song song/phân tán là gì?
- Có cần đầu tư nghiên cứu một mô hình tính toán mới không?

Nguyễn Ngọc Điệp

Những thuật ngữ quan trọng

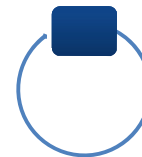
- Khả năng mở rộng (Scalability)
 - Khả năng đáp ứng được **sự tăng trưởng** của dữ liệu, tính toán và độ phức tạp mà không cần **hy sinh các dịch vụ và chức năng cốt lõi của hệ thống**
- Hiệu năng vào ra (I/O) của dữ liệu
 - **Tốc độ** dữ liệu được **truyền vào/lấy ra** từ **thiết bị ngoại vi**
- Khả năng chịu lỗi
 - Khả năng **duy trì hoạt động** chính xác khi **có lỗi xảy ra** tại một hoặc nhiều thành phần



Nguyễn Ngọc Điệp

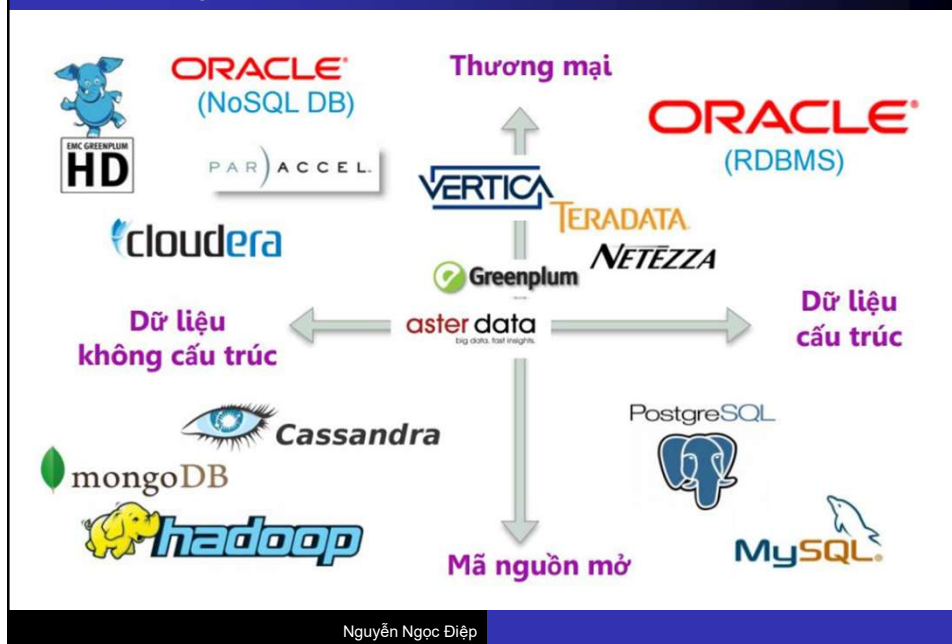
Những thuật ngữ quan trọng

- Xử lý thời gian thực
 - Khả năng **xử lý** dữ liệu và đưa ra kết quả chính xác trong **khoảng thời gian cho trước**
- Kích cỡ dữ liệu được hỗ trợ
 - **Kích cỡ** của **tập dữ liệu** mà hệ thống có thể xử lý và tính toán **hiệu quả**
- Hỗ trợ các tác vụ lặp
 - Khả năng hệ thống **hỗ trợ các tác vụ lặp** hiệu quả



Nguyễn Ngọc Điệp

Quản lý dữ liệu lớn



Xử lý dữ liệu lớn

- Công nghệ quan trọng để xử lý dữ liệu lớn
 - Mở rộng quy mô (scale out)
 - Tính toán phân tán
 - Tính toán song song
- Các công nghệ mã nguồn mở quan trọng
 - MapReduce
 - Hadoop
 - Spark
 - TensorFlow



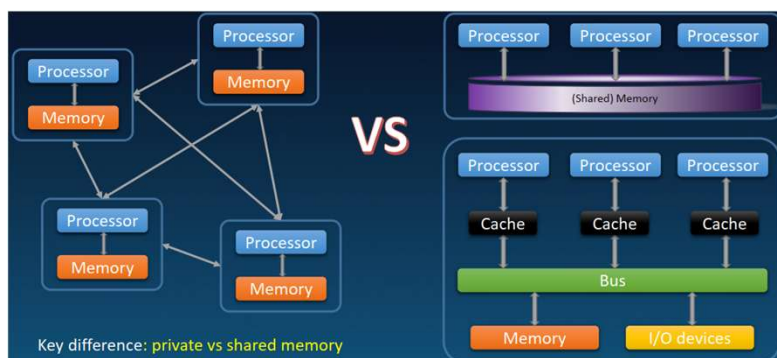
Tính toán phân tán và song song

Nguyễn Ngọc Diệp

Tính toán phân tán và song song

Tính toán phân tán: bài toán được chia nhỏ thành cụm và phân tán vào nhiều máy khác nhau; **mỗi máy có một bộ nhớ riêng.**

Tính toán song song: bài toán có cấu trúc tính toán song song, được chia nhỏ vào nhiều bộ xử lý để tính song song **có cùng bộ nhớ chung.**



Nguyễn Ngọc Diệp

Tính toán phân tán và song song

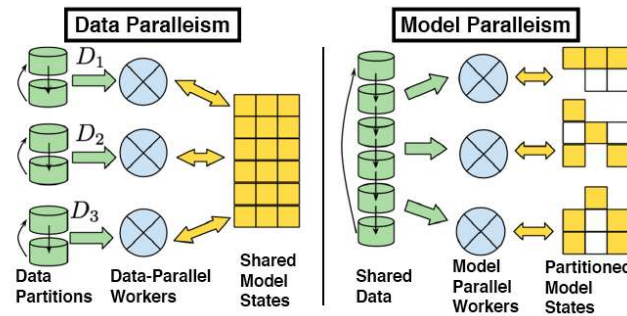
Phân tích dữ liệu = Mô hình + Dữ liệu

Song song hóa dữ liệu (data parallelism):

Dữ liệu được chia thành cụm và chạy song song trên cùng một mô hình

Song song hóa mô hình (model parallelism):

Nhiều mô hình chạy trên cùng một tập dữ liệu (e.g., Gibbs sampling)



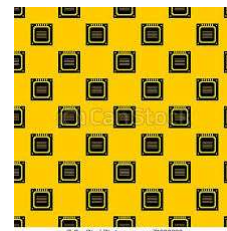
Nguyễn Ngọc Diệp

Tính toán phân tán và song song

Tính toán song song bằng phần cứng với GPU và CPU đa nhân

• CPU đa nhân

- Máy tính với hàng chục nhân xử lý
- Tính toán **song song** nhờ vào công nghệ **đa luồng** (multithreading)
- **Điểm yếu:**
 - Số nhân xử lý bị **hạn chế**
 - Bộ nhớ bị **hạn chế** ở tầm vài trăm GB

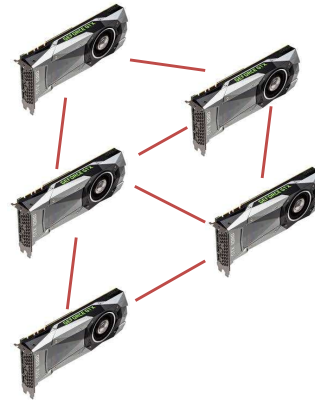


Nguyễn Ngọc Diệp

Tính toán phân tán và song song

Tính toán song song bằng phần cứng với GPU và CPU đa nhân

- GPU
 - ❑ Các bộ xử lý đơn giản với khả năng xử lý song song mức cao
 - ❑ Số nhân lớn (trên 2 nghìn)
 - ❑ Khả năng tăng tốc độ có thể so với CPU đa nhân
 - ❑ Điểm yếu:
 - Bộ nhớ bị hạn chế ở tầm 12 GB/GPU
 - Ít phần mềm và thuật toán cho GPU

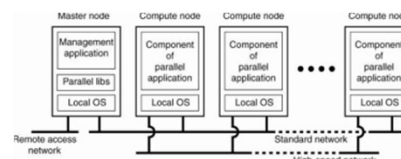


Nguyễn Ngọc Điệp

Tính toán phân tán và song song

Xử lý phân tán với hệ thống cluster

- Hệ thống tính toán cụm
 - ❑ Tập hợp các máy trạm hoặc PC tương tự nhau, được kết nối chặt chẽ bởi mạng LAN tốc độ cao, các nút chạy cùng một hệ điều hành
- Ưu điểm
 - ❑ Kinh tế: rẻ hơn 15 lần so với các siêu máy tính truyền thống có cùng hiệu năng
 - ❑ Khả năng mở rộng: Dễ dàng nâng cấp và bảo trì
 - ❑ Độ tin cậy: tiếp tục hoạt động ngay cả trong trường hợp hỏng một phần
- Nhược điểm
 - ❑ Khó quản lý và tổ chức một lượng lớn máy tính
 - ❑ Hiệu suất vào ra (I/O) của dữ liệu thấp
 - ❑ Không phù hợp để xử lý thời gian thực



Nguyễn Ngọc Điệp

Tính toán phân tán và song song

Xử lý phân tán trên cloud

- Được cung cấp bởi các công ty CNTT lớn
 - Nền tảng đám mây của Google
 - Dịch vụ web Amazon
 - Microsoft Azure
- Ưu điểm
 - chi phí đầu tư và duy trì **thấp**
 - bất cứ nơi nào và bất cứ lúc nào **có thể truy cập**
 - Khả năng mở rộng cao
- Nhược điểm
 - **bảo mật** dữ liệu
 - **phụ thuộc** vào nhà cung cấp
 - kết nối **internet** liên tục
 - vấn đề **di chuyển** hệ thống



Nguyễn Ngọc Diệp

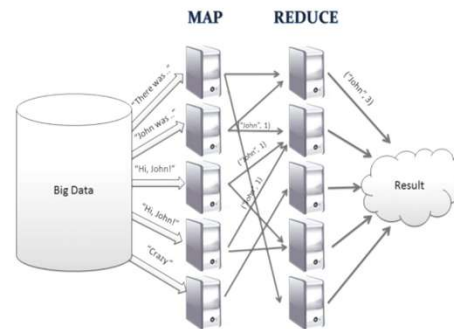
Giới thiệu một số công nghệ quan trọng

MapReduce, Hadoop, Spark, TensorFlow, MLlib

Nguyễn Ngọc Diệp

Công nghệ MapReduce

- Nhu cầu xử lý DLL tăng nhanh
- MapReduce
 - được Google phát minh vào năm 2004 và được sử dụng trong Hadoop
 - chia toàn bộ nhiệm vụ thành hai phần: mapper và reducer.
 - mapper: đọc dữ liệu từ HDFS, xử lý dữ liệu và tạo một số kết quả trung gian.
 - reducer: tổng hợp các kết quả trung gian để tạo kết quả đầu ra cuối cùng.
- Hạn chế chính
 - không hiệu quả trong việc **chạy các thuật toán lặp**, do Mapper **đọc cùng một dữ liệu nhiều lần** từ đĩa sẽ không hiệu quả



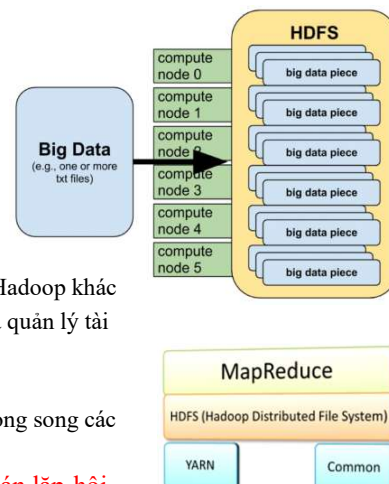
Data parallelization **Distributed execution** **Outcome aggregation**

[Nguồn: <https://www.supinfo.com/articles/single/2807-introduction-to-the-mapreduce-life-cycle>]

Nguyễn Ngọc Diệp

Công nghệ Apache Hadoop

- Apache Hadoop:
 - một khung nguồn mở để lưu trữ và xử lý các bộ dữ liệu lớn bằng cách sử dụng các cụm phần cứng thông thường
 - chịu lỗi cao
 - khả năng mở rộng lên 100 hoặc 1000 nút
- Thành phần Hadoop:
 - Common**: các tiện ích hỗ trợ các mô-đun Hadoop khác
 - YARN**: một khung để lập lịch công việc và quản lý tài nguyên cụm.
 - HDFS**: một hệ thống tệp phân tán
 - MapReduce**: mô hình tính toán để xử lý song song các bộ dữ liệu lớn.
- Hạn chế chính: **không phù hợp với thuật toán lặp-hội tụ!**



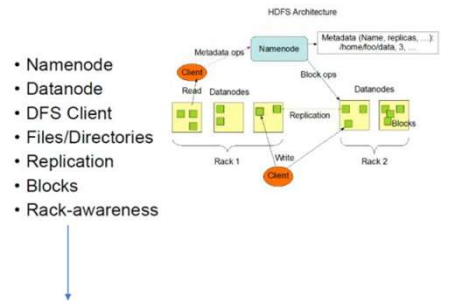
Nguyễn Ngọc Diệp

Công nghệ Apache Hadoop

- Hadoop Distributed File System (HDFS)

- Hệ thống tệp phân tán lưu trữ dữ liệu trên các máy tính thường, cung cấp **băng thông tổng hợp rất cao trên toàn cụm**
- được **thiết kế để xử lý dữ liệu phân tán quy mô lớn** theo các khung làm việc như MapReduce.
- **lưu trữ dữ liệu lớn** (ví dụ: 100TB) dưới dạng một tệp (chỉ cần xử lý một tệp duy nhất)
- **khả năng chịu lỗi**: mỗi khối dữ liệu được sao chép qua các DataNode. Sự dư thừa dữ liệu cho phép Hadoop phục hồi nếu một nút bị lỗi
-> gợi nhớ đến kiến trúc RAID

HDFS Terminology



"With a rack-aware file system, the JobTracker knows which node contains the data, and which other machines are nearby. If the work cannot be hosted on the actual node where the data resides, priority is given to nodes in the same rack. This reduces network traffic on the main backbone network".

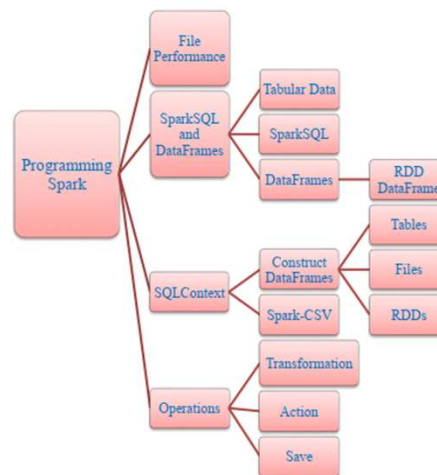
Nguyễn Ngọc Diệp

Công nghệ Apache Spark

- Động lực chính: **thích hợp cho các thuật toán lặp – hội tụ!**

- Các tính năng chính của Spark:

- Bộ dữ liệu phân tán linh hoạt (RDD)
 - Chỉ đọc, phân vùng bộ sưu tập các bản ghi được phân phối trên cụm, được lưu trữ trong bộ nhớ hoặc đĩa.
 - Xử lý dữ liệu = đồ thị của các biến đổi trong đó các nút = RDD và các cạnh = biến đổi.
- Lợi ích:
 - **Chịu lỗi**: có khả năng phục hồi cao do RDDs
 - **Khả năng cache**: lưu trữ một số RDD trong RAM, do đó nhanh hơn Hadoop MR để lặp lại.
 - Hỗ trợ MapReduce.



Nguyễn Ngọc Diệp

So sánh Spark và MapReduce




**WHY
Can't It Be
Even
Simpler?**




**What
Happiness
Looks
Like**

```

class RandomSplitRDD<prev: RDD[Array[Double]], seed: Long>
  extends RDD[Array[Double]](prev) {
    override def getPartitions: Array[Partition] = {
      val r = new Random(seed)
      Partitions(Array[Double]).partitions.map(x => new
        RandomSplitRDD(x, r))
    }
    override def getPreferredLocations(split: Partition):
      Seq[String] = {
        firstParent[Array[Double]].preferredLocations(split)
      }
    override def compute(splitIn: Partition, context: TaskContext,
      Iterator[Array[Double]]): Iterator[Array[Double]] = {
      val split = splitIn.asInstanceOf[RandomSplitRDD]
      val rand = new Random(split.seed)
      if (split.isFinal) {
        Iterator(Array[Double])
      } else {
        Iterator(Array[Double])
      }
    }
  }

def randomSplitRDD(prev: RDD[Array[Double]], seed: Long,
  trainingSize: Int, testSize: Int, shuffle: Boolean): RDD[Array[Double]] = {
  RDD[Array[Double]](new RandomSplitRDD(prev, seed, trainingSize,
    testSize, shuffle))
}

```

```

table = load("housePrices")
table.dropNA
table.train glm("price", "sf,zip,beds,baths")

```

[Nguồn: adato]

Nguyễn Ngọc Diệp

So sánh Spark và Hadoop

Sắp xếp 100 TB dữ liệu trên đĩa trong 23 phút; Kỷ lục thế giới trước đây được thiết lập bởi Hadoop MapReduce đã sử dụng 2100 máy và mất 72 phút.

Điều này có nghĩa là Apache Spark đã sắp xếp cùng một dữ liệu nhanh hơn 3 lần với số lượng máy ít hơn 10 lần.

	Hadoop MR Record	Spark Record	Spark 1 PB
Data Size	102.5 TB	100 TB	1000 TB
Elapsed Time	72 mins	23 mins	234 mins
# Nodes	2100	206	190
# Cores	50400 physical	6592 virtualized	6080 virtualized
Cluster disk throughput	3150 GB/s (est.)	618 GB/s	570 GB/s
Sort Benchmark Daytona Rules	Yes	Yes	No
Network	dedicated data center, 10Gbps	virtualized (EC2) 10Gbps network	virtualized (EC2) 10Gbps network
Sort rate	1.42 TB/min	4.27 TB/min	4.27 TB/min
Sort rate/node	0.67 GB/min	20.7 GB/min	22.5 GB/min

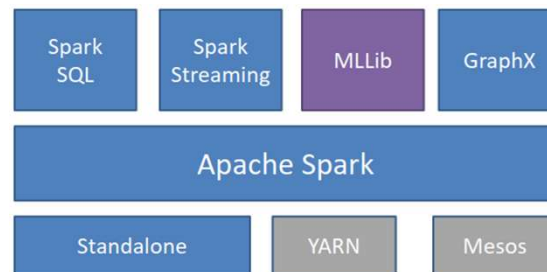
[Nguồn: <https://databricks.com>]

Nguyễn Ngọc Diệp

Mô hình lớn cho dữ liệu lớn

Mở rộng ML và các mô hình thống kê

- Hầu hết các thuật toán học máy và thống kê là **lặp - hội tụ!**
- Điều này là do hầu hết trong số chúng là các phương pháp dựa trên tối ưu hóa (ví dụ: Coordinate Descent, SGD) hoặc thuật toán suy luận thống kê (ví dụ: MCMC, SVI).
- Và, các thuật toán này về **bản chất được xây dựng theo kiểu lặp!**



Nguyễn Ngọc Điệp

Mô hình lớn cho dữ liệu lớn

Mở rộng ML và các mô hình thống kê

- **Lịch sử MLLib**
 - Một nền tảng trên Spark cung cấp các thuật toán mô hình thống kê và học máy có thể mở rộng.
 - Được phát triển từ AMPLab, UC Berkeley và được tích hợp vào Spark từ năm 2013.
- **Thuật toán Mllib**
 - **Classification**
 - Linear models (linear SVMs, logistic regression)
 - Naïve Bayes
 - Least squares
 - Classification tree
 - Ensembles of trees (Random Forests and Gradient-Boosted Trees)
 - **Regression**
 - Generalized linear models (GLMs)
 - Regression tree
 - Isotonic regression
 - **Clustering**
 - K-means
 - Gaussian mixture
 - Power iteration clustering (PIC)
 - Latent Dirichlet Allocation (LDA)
 - Streaming k-means
 - **Collaborative filtering (recommender system)**
 - Alternating least squares (ALS)
 - Non-negative matrix factorization (NMF)
 - **Dimensionality reduction**
 - Singular value decomposition (SVD)
 - Principal component analysis (PCA)
 - **Optimization**
 - SGD, L-BFGS

Nguyễn Ngọc Điệp

Mô hình lớn cho dữ liệu lớn

Mở rộng ML và các mô hình thống kê

- Mô hình học máy và thống kê cho dữ liệu lớn
 - ❑ Tại sao các phương pháp truyền thống có thể thất bại trên dữ liệu lớn?
 - ❑ Ba cách tiếp cận để mở rộng các mô hình lớn
 - Mở rộng Gradient Descent Methods
 - Mở rộng Probabilistic Inference
 - Mở rộng đánh giá mô hình (bootstrap)
 - ❑ Nguồn mở cho các mô hình lớn
 - Mlib, Tensorflow
- Ba cách tiếp cận để xây dựng các mô hình lớn của riêng bạn
 - ❑ Tăng dữ liệu
 - ❑ Stochastic Variational Inference cho các mô hình đồ họa
 - ❑ Stochastic Gradient Descent và học trực tuyến

Nguyễn Ngọc Diệp

Tóm tắt những thông điệp chính

- **Dữ liệu lớn là gì?**
 - Dữ liệu từ đâu đến?
 - Cơ hội và thách thức của DLL
- **Tiếp cận dữ liệu lớn như thế nào?**
 - Quản lý dữ liệu lớn
 - Xử lý dữ liệu lớn
 - Tính toán phân tán và song song
 - Giới thiệu nền tảng công nghệ
- DLL ngày càng phổ biến và có ba đặc tính quan trọng (3V):
 - ❑ Kích thước rất lớn
 - ❑ Dữ liệu đa dạng khó điều khiển
 - ❑ Dòng dữ liệu không ngừng chuyển động.
- DLL đem lại nhiều cơ hội nhưng cũng đặt ra nhiều thách thức.
- Khi tiếp cận DLL có ba bước ta cần quan tâm:
 - ❑ Lưu trữ dữ liệu như thế nào?
 - ❑ Dùng công nghệ gì để xử lý dữ liệu?
 - ❑ Dùng công cụ và mô hình gì để phân tích dữ liệu?
- Những nền tảng công nghệ mã nguồn mở chính cho DLL để tham khảo bao gồm:
 - ❑ Hadoop + MapReduce
 - ❑ Spark
 - ❑ TensorFlow (chủ yếu cho deep learning)

Nguyễn Ngọc Diệp